



Semana 5 - Recursão

Problema A - The Coca-Cola Store

Definiremos a função recursiva *coca_cola* da seguinte maneira:

$$coca_cola(x) = \begin{cases} 0, & x \leq 1 \\ 1, & x = 2 \\ n/3 + coca_cola(n/3 + (n\%3)), & \text{caso contrário} \end{cases}$$

onde a função retornará 0 no caso de 1 ou nenhuma garrafa, 1 no caso de 2 (visto que o problema fala que podemos pegar uma garrafa vazia da loja para completar 3 garrafas). No último caso, recebemos $\frac{n}{3}$ garrafas cheias, as consumimos, e então chamamos a função novamente para as garrafas que acabamos de consumir + as que não foram trocadas.

Complexidade: $\approx O(\log_3(n))$

```
#include "stdio.h"
#include "stdlib.h"

int coca_cola(int n) {
    if(n <= 1) return 0;
    if(n == 2) return 1;

    int ret = n/3;
    return ret + coca_cola(n/3 + ret);
}

int main() {
    int n; scanf("%d",&n);

    while(n) {
        int ans = coca_cola(n);

        printf("%d\n",ans);

        scanf("%d",&n);
    }
}
```

Problema B - Maximize?

Como a entrada do problema é pequena, basta testarmos cada valor de y ($2 \leq y < x$). Para calcular o Máximo Divisor Comum (MDC), utilizaremos o algoritmo recursivo euclidiano para achar o MDC:

$$mdc(a,b) = \begin{cases} a, & b = 0 \\ mdc(b, a \bmod b), & \text{caso contrário} \end{cases}$$

Complexidade: $O(t \times x \times \log \min(a, b))$

```
#include "stdio.h"
#include "stdlib.h"
```



```
int gcd(int a, int b) {
    return b ? gcd(b, a%b) : a;
}

int main() {
    int t; scanf("%d",&t);
    while(t--) {
        int x; scanf("%d",&x);

        int max = 0, ans = -1;
        for(int y = 1; y < x; y++) {
            int mdc = gcd(x, y);

            if(mdc+y > max) {
                max = mdc+y;
                ans = y;
            }
        }

        printf("%d\n", ans);
    }
}
```

Problema C - Computer Game

Para este problema, basta verificarmos se há no mínimo um 0 em cada coluna da matriz.

Complexidade: $O(n)$

```
#include "stdio.h"
#include "stdlib.h"

int main() {
    int t; scanf("%d",&t);
    while(t--) {
        int n; scanf("%d",&n);

        char mat[2][n+1];
        for(int i = 0; i < 2; i++) scanf("%s", mat[i]);

        int ans = 1;
        for(int i = 0; i < n; i++) ans &= !((mat[0][i] - '0') && (mat[1][i] - '0'));

        if(ans) printf("YES\n");
        else printf("NO\n");
    }
}
```

Problema D - Gold Rush

Queremos saber se é possível formar uma pilha de tamanho m a partir da separação de uma pilha em duas, tal que uma das pilhas resultantes tenha o dobro de ouro do que a outra.

Para isso ser possível, a pilha n deve ser um múltiplo de 3. Seja, n a pilha inicial, p_1 e p_2 as pilhas resultantes da separação de n .

$$n = p_1 + p_2$$

Sabendo que p_2 deve ter o dobro de ouro de p_1 (sem perda de generalidade), logo $p_2 = 2 \times p_1$.



$$n = p_1 + 2 \times p_1$$

$$n = 3 \times p_1$$

Com isso, definiremos a função *rush* na qual:

$$rush(n) = \begin{cases} TRUE, & n = m \\ FALSE, & n \not\equiv 0 \pmod{3} \\ (rush(\frac{n}{3}) \text{ OR } rush(2 \times \frac{n}{3})), & \text{caso contrário} \end{cases}$$

Complexidade: $\approx O(\log_3(n))$

```
#include "stdio.h"
#include "stdlib.h"

int rush(int n, int m) {
    if(n == m) return 1;
    if(n % 3 != 0) return 0;

    return (rush(n/3,m) || rush(2*n/3,m));
}

int main() {
    int t; scanf("%d\n",&t);
    while(t--) {
        int n,m; scanf("%d-%d",&n,&m);

        if(rush(n,m)) printf("YES\n");
        else printf("NO\n");
    }
}
```

Problema E - Transformation: from A to B

Para transformar B em A , seguimos uma abordagem que utiliza o caminho inverso das operações permitidas. Sabemos que, se B for par, ele deve ter sido obtido multiplicando algum número por 2, então dividimos B por 2. Caso contrário, se B termina em 1 (ou seja, $B \% 10 = 1$), ele deve ter sido obtido ao adicionar 1 no final de um número, e, nesse caso, subtraímos 1 de B e o dividimos por 10. Repetimos esse processo até que B se torne igual a A .

À medida que transformamos B de volta para A , armazenamos os valores intermediários em uma array, representando o caminho que leva de B até A . Quando o processo termina, a array contém o caminho inverso. Finalmente, o programa imprime o tamanho da array, que corresponde ao número de operações necessárias, seguido pelos valores do caminho.

Complexidade: $\approx O(\max(\log b, \log_{10} b))$

```
#include "stdio.h"
#include "stdlib.h"

int ptr_fila = 0;
int fila[1000001];

int transform(int a, int b) {
    while(1) {
        if(b == a) {
            fila[ptr_fila] = b;
            ptr_fila++;

            return 1;
        } else if(b % 2 == 0 && b > 0) {
```



```
        fila[ptr_fila] = b;
        ptr_fila++;

        b /= 2;
    } else if ((b-1) % 10 == 0 && b > 0) {
        fila[ptr_fila] = b;
        ptr_fila++;

        b--;
        b /= 10;
    } else return 0;
}

int main() {
    int a,b; scanf("%d-%d",&a,&b);

    int ans = transform(a,b);
    if(!ans) printf("NO\n");
    else {
        printf("YES\n%d\n", ptr_fila);
        while(--ptr_fila >= 0) printf("%d-", fila[ptr_fila]);
        printf("\n");
    }
}
```

Problema F - Desafio: Prisões & Wyverns

O problema consiste em determinar a quantidade de formas diferentes de alcançar uma soma n com o lançamento de um número arbitrário de dados. Inicialmente, podemos abordar o problema com uma função recursiva que recebe um valor x , representando a soma desejada, e testa recursivamente as possibilidades ao lançar mais um dado. Para cada lançamento, consideramos todos os valores possíveis (de 1 a 6) e diminuímos esses valores de x . O caso base ocorre quando x é igual a zero, onde retornamos 1, pois encontramos uma combinação válida.

Embora essa abordagem seja conceitualmente correta, ela é computacionalmente ineficiente. O número de chamadas recursivas cresce exponencialmente, tornando o algoritmo extremamente lento para valores maiores de n . Como resultado, essa solução ultrapassaria os limites de tempo da questão.

```
int busca(int x) {
    if(x == 0) return 1;

    int ret = 0;
    for(int dado = 1; dado <= 6; dado++) ret += busca(x-dado) % MOD;

    return ret;
}
```

O problema com a abordagem inicial não está na lógica, mas na falta de otimização. A solução para isso é aplicar o paradigma de **programação dinâmica**, que reutiliza valores previamente calculados para evitar repetições desnecessárias, tornando o algoritmo mais eficiente.

Nesse caso, podemos introduzir uma array chamada *tab*, onde armazenamos os resultados das operações realizadas para um determinado valor x , que representa quanto ainda falta para atingir a soma desejada. Quando a função recursiva, chamada de *dp* (em referência a "dynamic programming"), é usada para calcular o resultado de x , verificamos se o valor já foi calculado anteriormente. Se for o caso, simplesmente retornamos o valor armazenado no *tab*. Caso contrário, realizamos o cálculo recursivamente e armazenamos o resultado no *tab* para reutilizações futuras.

Esse método elimina redundâncias, reduz o número de chamadas recursivas e acelera significativamente o programa, tornando-o capaz de lidar com maiores valores de n sem exceder os limites de tempo.

Complexidade: $O(n)$



```
#include "stdio.h"
#include "stdlib.h"

int tab[3000];

int const MOD = 1000000007;

int dp(int n) {
    if(n < 0) return 0;
    if(tab[n] != -1) return tab[n];

    int ret = 0;
    for(int d = 1; d <= 6; d++) ret = (ret + dp(n-d)) % MOD;

    return tab[n] = ret;
}

int main() {
    int n; scanf("%d",&n);

    for(int i = 0; i < 3000; i++) tab[i] = -1;
    tab[0] = 1;

    printf("%d\n",dp(n));
}
```